

Recipe Intelligence Platform — Algorithm Guide

How to Implement All 12 Algorithms & Select the Best Model

How This Guide Is Organized

Each of the 4 team members implements 3 algorithms within their assigned category. Every algorithm below includes: what it does, the code to run it, and which metric to record for the final comparison in Section 5, where we determine the single best-performing model across the whole project.

Y (Target) = rating_binary (1 = rating ≥ 4 “Good”, 0 = rating < 4 “Poor”) — 88.6% / 11.4% split

MEMBER 1 — Classification

1.1 Logistic Regression

A linear baseline model that estimates the probability of a recipe being rated 'Good' using a weighted sum of features passed through a sigmoid function.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score, f1_score

log_model = LogisticRegression(class_weight='balanced', max_iter=1000)
log_model.fit(X_train, y_train)

y_pred = log_model.predict(X_test)
y_proba = log_model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
print("F1 Score:", f1_score(y_test, y_pred))
```

1.2 Random Forest

An ensemble of decision trees, each trained on a random subset of data/features; the majority vote across trees gives the final prediction. Handles mixed feature types well without scaling.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(n_estimators=200, class_weight='balanced', random_state=42)
rf_model.fit(X_train, y_train)

y_pred = rf_model.predict(X_test)
y_proba = rf_model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
```

1.3 XGBoost

A gradient boosting ensemble that builds trees sequentially, each correcting the errors of the previous one — typically the strongest performer among tree-based classifiers.

```
from xgboost import XGBClassifier

scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
xgb_model = XGBClassifier(scale_pos_weight=scale_pos_weight, random_state=42, eval_metric='logloss')
xgb_model.fit(X_train, y_train)

y_pred = xgb_model.predict(X_test)
y_proba = xgb_model.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
```

Metric to record:

F1-Score and ROC-AUC for all three models (never plain Accuracy, since the classes are imbalanced 88.6% / 11.4%).

2.1 Naive Bayes (TF-IDF baseline)

A probabilistic classifier assuming word-independence; fast, simple, and a strong baseline specifically for text/TF-IDF features such as review and ingredients text.

```
from sklearn.naive_bayes import MultinomialNB

nb_model = MultinomialNB()
nb_model.fit(X_train_text, y_train) # X_train_text = TF-IDF matrix only (non-negative values)

y_pred = nb_model.predict(X_test_text)
print(classification_report(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))
```

2.2 LSTM

A recurrent neural network that processes review/ingredient text sequentially, retaining context across words — captures word order, which TF-IDF cannot.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

tokenizer = Tokenizer(num_words=10000)
tokenizer.fit_on_texts(df_clean['review'])
sequences = tokenizer.texts_to_sequences(df_clean['review'])
padded = pad_sequences(sequences, maxlen=100)

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 64, input_length=100),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(padded, y, epochs=3, validation_split=0.2, class_weight={0: 7.8, 1: 1.0})
```

2.3 BERT (Fine-tuned)

A pre-trained transformer fine-tuned on review text; understands context and word relationships far better than TF-IDF or LSTM, at higher compute cost.

```
from transformers import BertTokenizer, TFBertForSequenceClassification
import tensorflow as tf

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)

# Tokenize a SAMPLE (BERT is expensive – use a subset, e.g. 20,000 rows, given 1.13M total rows)
sample = df_clean.sample(20000, random_state=42)
encodings = tokenizer(list(sample['review']), truncation=True, padding=True, max_length=128,
return_tensors='tf')

bert_model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=2e-5),
loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True))
bert_model.fit(dict(encodings), sample['rating_binary'], epochs=2, batch_size=16)
```

Metric to record:

F1-Score for all three; additionally compare training time and dataset size used, since BERT/LSTM are run on a sample while Naive Bayes runs on the full set — note this tradeoff explicitly when comparing.

MEMBER 3 — Clustering (Unsupervised)

3.1 PCA (run first — feeds into both clustering methods below)

Reduces high-dimensional TF-IDF features (ingredients/tags) into a smaller set of components that retain most of the variance, making clustering faster and more stable.

```
from sklearn.decomposition import PCA

pca = PCA(n_components=50, random_state=42)
reduced_features = pca.fit_transform(ingredients_vec.toarray())
print("Explained variance:", sum(pca.explained_variance_ratio_))
```

3.2 K-Means

Partitions recipes into K clusters by minimising distance to each cluster's centroid — fast and scalable for 1M+ rows.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Use a sample for the K search (silhouette is expensive on 1M+ rows)
sample_idx = df_clean.sample(20000, random_state=42).index
sample_features = reduced_features[sample_idx]

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(sample_features)
    score = silhouette_score(sample_features, labels)
    print(f"K={k}: Silhouette={score:.3f}")

# Fit final model with chosen K on full data
final_kmeans = KMeans(n_clusters=6, random_state=42, n_init=10)
df_clean['cluster'] = final_kmeans.fit_predict(reduced_features)
```

3.3 Hierarchical Clustering

Builds a tree of nested clusters by iteratively merging the closest pairs — used here to validate K-Means groupings on a sample (too slow for the full 1.13M rows).

```
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

sample_features = reduced_features[:5000] # hierarchical clustering is O(n^2) – sample only

agglo = AgglomerativeClustering(n_clusters=6)
agglo_labels = agglo.fit_predict(sample_features)

score = silhouette_score(sample_features, agglo_labels)
print("Hierarchical Silhouette Score:", score)
```

Metric to record:

Silhouette Score for K-Means and Hierarchical Clustering; Explained Variance Ratio for PCA.

MEMBER 4 — Recommendation System

4.1 Content-Based Filtering

Recommends recipes similar in ingredients/tags to a recipe the user liked, using cosine similarity on TF-IDF vectors — works even for brand-new recipes with no ratings (solves cold-start).

```
from sklearn.metrics.pairwise import cosine_similarity

def recommend_similar(recipe_index, top_n=5):
    sims = cosine_similarity(ingredients_vec[recipe_index], ingredients_vec).flatten()
    similar_indices = sims.argsort()[::-1][1:top_n+1]
    return df_clean.iloc[similar_indices][['name', 'rating']]

print(recommend_similar(0))
```

4.2 Collaborative Filtering

Recommends recipes based on what similar USERS rated highly, using a user-item interaction matrix — captures taste patterns content-based filtering cannot.

```
from surprise import Dataset, Reader, KNNBasic
from surprise.model_selection import train_test_split as surprise_split

reader = Reader(rating_scale=(0, 5))
data = Dataset.load_from_df(df_clean[['user_id', 'recipe_id', 'rating']], reader)
trainset, testset = surprise_split(data, test_size=0.2)

cf_model = KNNBasic(sim_options={'user_based': True})
cf_model.fit(trainset)

predictions = cf_model.test(testset)
from surprise import accuracy
accuracy.rmse(predictions)
```

4.3 Matrix Factorization (SVD)

Decomposes the sparse user-recipe rating matrix into latent factors, capturing hidden taste dimensions — generally the strongest classical recommender approach.

```
from surprise import SVD

svd_model = SVD(n_factors=50, random_state=42)
svd_model.fit(trainset)

predictions = svd_model.test(testset)
accuracy.rmse(predictions)
accuracy.mae(predictions)
```

Metric to record:

RMSE and MAE for Collaborative Filtering and SVD; for Content-Based Filtering, evaluate qualitatively (do recommended recipes share genuine ingredient/tag overlap) since it has no direct rating-error metric.

5. How to Evaluate and Select the Best Model

5.1 Metrics Are Different by Category — You Cannot Compare Across Categories Directly

Classification models (Member 1 & 2) are compared using F1-Score and ROC-AUC. Clustering (Member 3) has no “accuracy” — it is compared using Silhouette Score. Recommendation (Member 4) is compared using RMSE/MAE. The “best model” is therefore chosen WITHIN each category first, then the four category winners are presented together as your final pipeline — not squeezed into one single ranking.

5.2 Within-Category Comparison Table Template

Model	Metric	Score
Logistic Regression	F1 / ROC-AUC	Fill in after running
Random Forest	F1 / ROC-AUC	Fill in after running
XGBoost	F1 / ROC-AUC	Fill in after running

5.3 Rule for Picking the Winner in Each Category

Category	Selection Rule
Classification (Member 1 & 2)	Highest F1-Score wins. If two models tie on F1, prefer the higher ROC-AUC — it shows better separation between classes across all thresholds, not just the default 0.5 cutoff.
Clustering (Member 3)	Highest Silhouette Score wins (closer to +1 = tighter, better-separated clusters). Cross-check that K-Means and Hierarchical Clustering broadly agree on cluster count/structure before trusting the result.
Recommendation (Member 4)	Lowest RMSE/MAE wins for Collaborative Filtering vs SVD. Content-Based Filtering is kept regardless of its ranking here, since it uniquely solves cold-start (new recipes with zero ratings) — the other two cannot.

5.4 Common Pitfall to Avoid

- **Do not use plain Accuracy** for the classification models — with an 88.6% / 11.4% split, a model that always predicts ‘Good’ scores 88.6% accuracy while learning nothing. F1-Score and ROC-AUC expose this immediately.
- **Do not compare BERT/LSTM directly against Naive Bayes on raw score alone** without noting BERT/LSTM were trained on a smaller sample (compute cost) — mention this tradeoff explicitly when presenting.
- **Do not silence Content-Based Filtering's weaker RMSE** — it is not meant to compete on rating-prediction accuracy; its value is solving cold-start, a capability the others lack entirely.

5.5 Final Recommended Pipeline (What You Present Tomorrow)

Pipeline Stage	Model Used
Primary rating prediction	Best of: Logistic Regression / Random Forest / XGBoost (by F1-Score)
Text-only rating signal	Best of: Naive Bayes / LSTM / BERT (by F1-Score, noting sample-size tradeoff)

Pipeline Stage	Model Used
Recipe grouping	K-Means (validated against Hierarchical Clustering via Silhouette Score)
Recommendation engine	SVD or Collaborative Filtering (by RMSE) + Content-Based Filtering (for cold-start)

— End of Document —